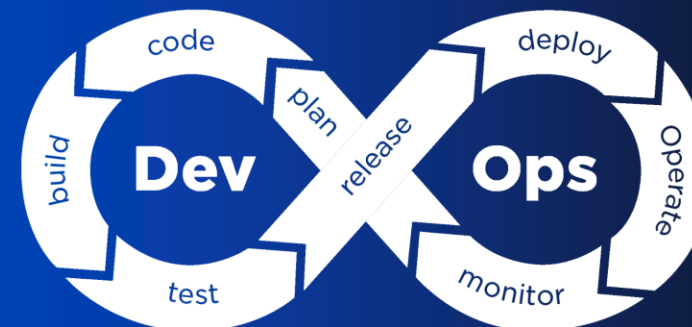


Introdução ao DevOps

Semana 1

CSDV-246 - DevOps



Conteúdo

- A Cultura dos Silos
- Metodologias Ágeis
- Conceitos de DevOps
- Cultura e Colaboração
- Práticas de DevOps
- Valores de DevOps
- Áreas de Impacto de DevOps
- Mal-entendidos sobre DevOps
- Estratégias de *Branching*



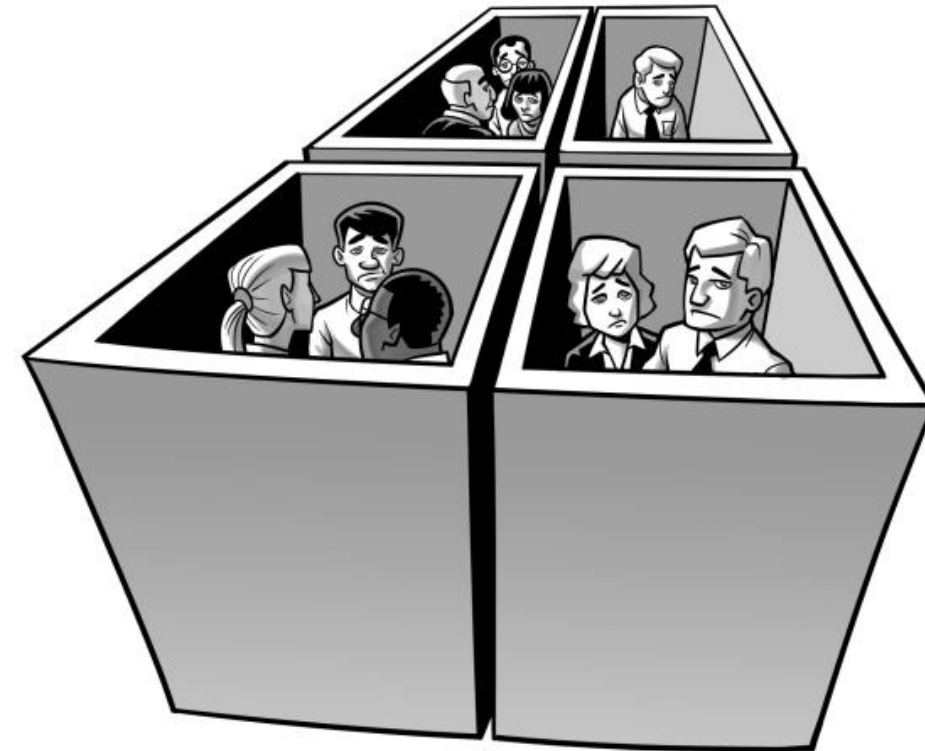
A Cultura dos Silos

A Cultura dos Silos



Silo

- Isolar-se dos outros
- Quando determinados departamentos ou setores da empresa não conseguem, ou não sabem, compartilhar informações, sistemas ou processos com outros da mesma empresa.



O que uma Empresa ou Negócio querem



- Chegar rapidamente ao mercado.
- Maximizar o Retorno sobre Investimento (ROI).
- Otimizar recursos.
- Processos flexíveis de Desenvolvimento de Software.
- Resposta rápida a incidentes.
- Feedback rápido do cliente.
- Tomada de decisões.



Silos Típicos no Desenvolvimento de Software



Equipe de Desenvolvimento

- Criação de software.
- Tarefas de Programação.
- Testes e Depuração.
- Ciclo de Vida do Desenvolvimento de Software.



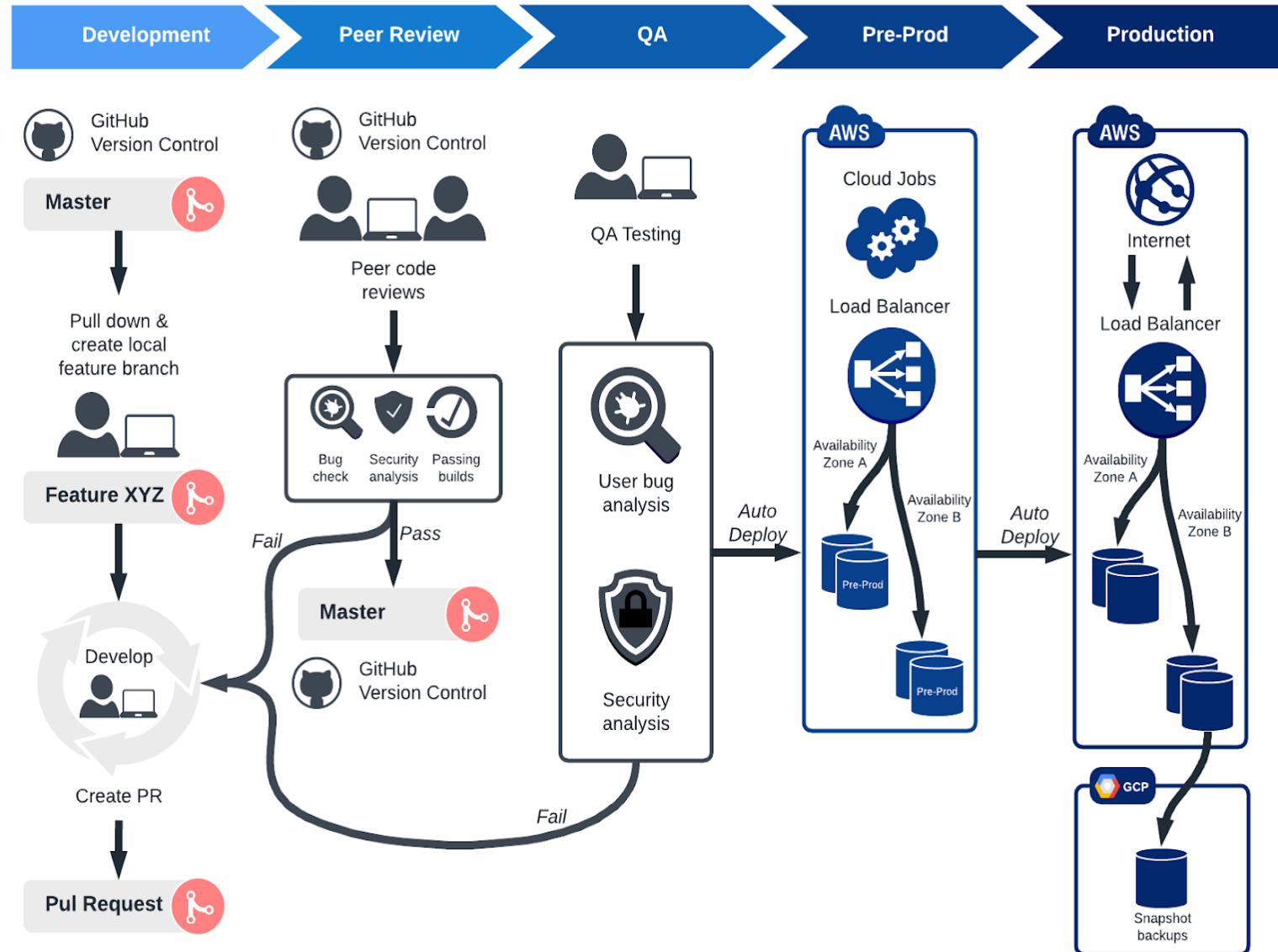
Equipe de Operações

- Infraestrutura e Monitoramento.
- Arquitetura e Planejamento.
- Manutenção.
- Suporte.



Metodologías Ágeis

Ciclo de vida de desenvolvimento de software



Metodologias de Desenvolvimento de Software

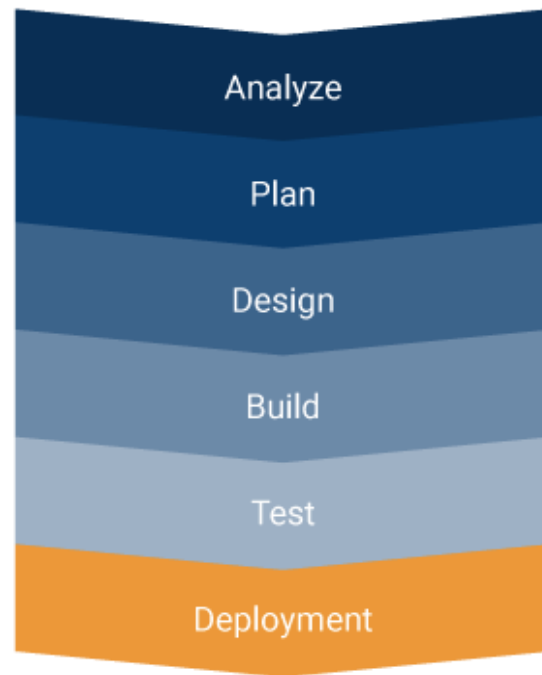


Tradicionais

Prototipação

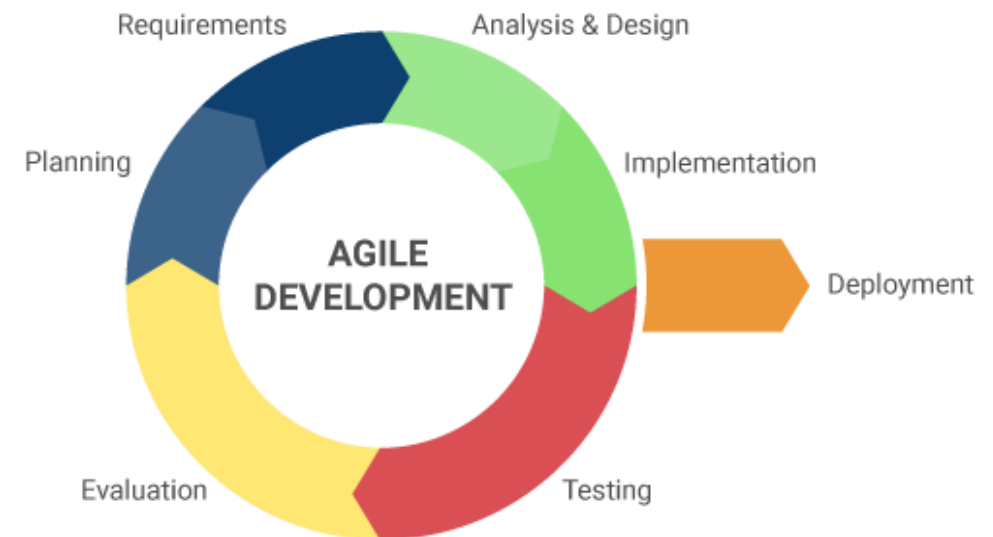
Cascata

Iterativo



VS.

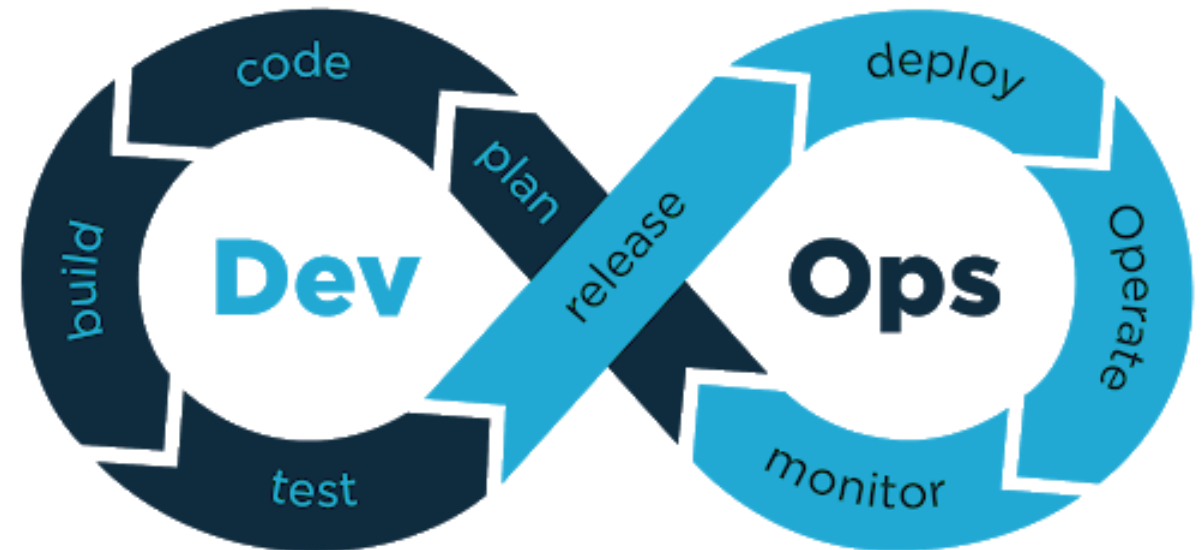
Ágil



Metodologias de Desenvolvimento de Software



- DevOps e Ágil não são a mesma coisa.
- O DevOps é baseado em metodologias ágeis.



Os conceitos de DevOps

Os conceitos de DevOps



Definição de Comunidade

"DevOps é um conjunto de práticas que combinam desenvolvimento de software (Dev) e operações de TI (Ops).

Seu objetivo é encurtar o ciclo de vida de desenvolvimento de sistemas e fornecer entrega contínua com software de alta qualidade.

O DevOps é complementar ao desenvolvimento de software ágil, vários aspectos do DevOps vêm da metodologia Agile."

[Wikipedia](#)

Os conceitos de DevOps



Definições da Indústria

“DevOps é a união de pessoas, processos e tecnologia para entregar valor continuamente aos clientes.”

Microsoft

“DevOps é a combinação de filosofias, práticas e ferramentas culturais que aumentam a capacidade de uma organização de fornecer aplicativos e serviços em alta velocidade. Para melhor atender seus clientes e competir de forma mais eficaz no mercado.”

Amazon

Os conceitos de DevOps



Definições de acadêmicos

"DevOps não é um objetivo, mas um processo interminável de melhoria contínua."

Jez Humble

"É todo o trabalho e esforço para deixar o software sempre em um estado estável e pronto para ser lançado. De forma que possamos levar continuamente ideias para a produção..."

Dave Farley

Cultura e colaboração

Cultura e colaboração



- Quebrando a cultura dos Silos.
- Melhorar a comunicação da equipe.
- Aplicação de Boas Práticas de Desenvolvimento.
- Ágil - Ciclos de Vida de Desenvolvimento Rápido de Software.
- Colaboração entre as equipes de Desenvolvimento e Operações de Software.
- Trabalho em equipe.
- Inovação e valor agregado.
- Compartilhar responsabilidades.

As Práticas de DevOps

As Práticas de DevOps



- Trata-se de um conjunto de abordagens e metodologias que combinam desenvolvimento de software e operações de TI para melhorar a colaboração, a automação e a entrega contínua de software.
- Essas práticas buscam acelerar o ciclo de desenvolvimento, aumentar a qualidade do software e melhorar a eficiência operacional.



As Práticas de DevOps

- Gerenciamento de configuração
- Infraestrutura como código (IaC)
- Integração Contínua (CI)
- Entrega Contínua (CD)
- Implantação Contínua
- Orquestração
- Monitoramento e observabilidade

Gerenciamento de configuração



- Garante que as equipes de desenvolvimento possam implantar, manter e monitorar ambientes com eficiência, o que é crucial em fluxos de trabalho ágeis de DevOps.
- Garante que os sistemas mantenham sua INTEGRIDADE ao longo do tempo.
- Envolve definir, manter e garantir a consistência no desempenho, atributos funcionais e físicos de um sistema por meio de ferramentas e processos automatizados.

Infraestrutura como código



- É a prática de gerenciar e provisionar a infraestrutura de TI por meio de arquivos de configuração, em vez de executar processos manuais.
- Ele trata a infraestrutura de forma semelhante ao código de software, permitindo que as equipes de desenvolvimento automatizem a implantação e o gerenciamento de recursos como servidores, bancos de dados e redes.

Integração Contínua (CI)



- É a prática em que os desenvolvedores mesclam regularmente suas alterações de código em um repositório central, várias vezes ao dia.
- Cada integração é testada e validada automaticamente por meio de compilações e testes automatizados, garantindo que as alterações no código sejam verificadas antecipadamente e com frequência.
- A CI melhora a qualidade do código, reduz os problemas de integração e acelera o ciclo de vida do desenvolvimento, garantindo que o novo código funcione bem no sistema existente desde os estágios iniciais.

Entrega Contínua (CD)



- É a prática de ter automaticamente as alterações de código prontas para implantação manual na produção depois de passarem por testes e validação automatizados rigorosos.
- Garante que o software esteja sempre pronto para produção. Assim, os desenvolvedores podem fornecer recursos, correções de bugs e atualizações de forma rápida e confiável.
- Reduz o tempo e o esforço necessários para implementar novos recursos e atualizações, permitindo ciclos de lançamento mais rápidos e mantendo altos padrões de qualidade.

Implantação Contínua



- É a prática de implantar automaticamente na produção, cada alteração de código que passou com êxito em todos os testes automatizados.
- É uma extensão da Entrega Contínua, pois automatiza totalmente o processo desde a Implantação até a Produção.
- Ele permite que os desenvolvedores obtenham a máxima eficiência no lançamento de atualizações de software, garantindo que o código mais recente esteja em produção o mais rápido possível, reduzindo o risco de implantações gigantes e complexas e melhorando a velocidade geral de entrega.

Orquestração



- Refere-se à coordenação e automação da execução de vários processos ou serviços em diferentes sistemas para gerenciar aplicativos e recursos em escala.
- Automatiza tarefas como implantação de contêineres, escalonamento, balanceamento de carga e monitoramento de integridade em um cluster de servidores.
- Ele também detecta falhas, erros em serviços ou contêineres e os reinicia automaticamente para minimizar o tempo de inatividade.

Monitoramento e observabilidade



- **Monitorização** refere-se ao processo de coleta, análise e visualização de dados de sistemas e aplicativos para garantir que eles estejam funcionando conforme o esperado, rastreando métricas específicas, como uso de CPU, memória, tráfego de rede, taxas de erro e tempos de resposta.
- **Observabilidade** vai além do monitoramento, pois fornece informações sobre o estado interno de um sistema em funcionamento. Ele permite uma compreensão mais profunda e a solução de problemas de sistemas complexos, mesmo quando o problema não é predefinido ou monitorado diretamente.

Valores do DevOps

Valores do DevOps



Melhore a experiência do cliente

- Serviços interrompidos custam vendas ou reputação às empresas.
- Obter *feedback* do cliente o mais rápido possível.
- Trabalho em Equipe.
- Capacidade de reagir de forma ágil.

Valores do DevOps



Aumentar a capacidade de inovação

- Reduzir o desperdício de tempo e o retrabalho.
- Otimizar recursos para gerar maior valor.
- Automação de processos.

Gere valor o mais rápido possível

- Desenvolver cultura, práticas e automação.
- Fornecer software de forma mais rápida, eficiente e confiável.

Áreas de impacto do DevOps

Áreas de Impacto de DevOps



Cultura de trabalho

As organizações precisam:

- Promover a colaboração entre as equipes.
- Compartilhar responsabilidades.
- Aprendizado contínuo.
- Aplicar boas práticas de desenvolvimento.
- Criar equipes multifuncionais ou multidisciplinares.

Áreas de impacto do DevOps



Functional

Common functional expertise



System analysts



Developers



Testers

Cross - Functional

Representatives from the various functions



Development Team

Áreas de impacto do DevOps



Administração de Gestão

As organizações precisam:

- Dimensionar a tecnologia atual ao longo do tempo.
- Dimensionar o tamanho da organização.
- Sobreviver à burocracia, reduzir ao máximo possível.
- Manter a liderança alinhada com as pessoas.
- Implementar modelos sólidos de Arquitetura Corporativa.

Áreas de impacto do DevOps



Área Técnica e Desenvolvimento

Como otimizar processos:

- Automatize o maior número possível de processos repetitivos.
- Desenvolva software com escalabilidade e manutenibilidade em mente.
- Aplique Infraestrutura como Código e Gerenciamento de Configuração.
- Habilite a observabilidade e o monitoramento para analisar dados de desempenho de produtos e infraestrutura.

Mal-entendidos sobre DevOps

Mal-entendidos sobre DevOps



- O DevOps envolve apenas desenvolvedores e operação.
- O DevOps é uma equipe independente.
- DevOps é um cargo.
- O DevOps só é relevante em Startups com produtos na Web.
- Uma certificação DevOps é necessária para implementá-lo na empresa.
- DevOps significa fazer todo o trabalho com metade da equip

Mal-entendidos sobre DevOps



- Levará uma quantidade X de semanas ou meses para implementar o DevOps.
- DevOps é apenas lidar com ferramentas.
- DevOps tem tudo a ver com automação.
- DevOps é uma moda passageira.
- Existe apenas um "Caminho Certo" (ou "Caminho Errado") para fazer DevOps.

Estratégias de ramificação



Estratégias de ramificação

- Essas são as estratégias que as equipes de desenvolvimento de software adotam ao escrever, mesclar e implantar código usando um Sistema de Controle de Versão (VCS).
- Basicamente, eles são um conjunto de regras, etapas, que os desenvolvedores podem seguir para determinar como vão interagir com uma base de código compartilhada.

Importância das estratégias de ramificação



- Uma estratégia é necessária, pois ajuda a manter os repositórios organizados para evitar bugs no aplicativo.
- Conflitos e problemas de mesclagem aparecem ao trabalhar em paralelo em uma equipe de desenvolvimento.
- Essas estratégias não estão "vinculadas" a uma plataforma ou ferramenta específica. Isso permite que eles sejam aplicados em diferentes contextos.

Importância das estratégias de ramificação



- Ao trabalhar em DevOps, devemos criar um processo rápido e ágil no qual pequenas atualizações de código possam ser lançadas com frequência.
- Esses conflitos podem retardar o processo de entrega rápida de código e interromper nossos fluxos de trabalho.



Estratégias de ramificação

Algumas das estratégias mais conhecidas e utilizadas:

- Git Flow
- GitHub Flow
- GitLab Flow

Git Flow



- É um modelo de ramificação antigo que ajuda a gerenciar o desenvolvimento de funcionalidades e novos lançamentos de forma estruturada.
- Define um fluxo de trabalho claro para que as equipes de desenvolvimento colaborem no código.
- Se sua equipe pratica a Entrega Contínua, é recomendável adotar uma estratégia muito mais simples, como o GitHub *Flow*.



Ramos principais (*branches* principais)

- *Master/Main*: O código pronto para produção deve conter apenas código estável que foi implantado no ambiente de produção.
- *Develop*: É a *branch* onde todos os novos recursos são integrados e testados antes de serem lançados para uma nova implantação.e.

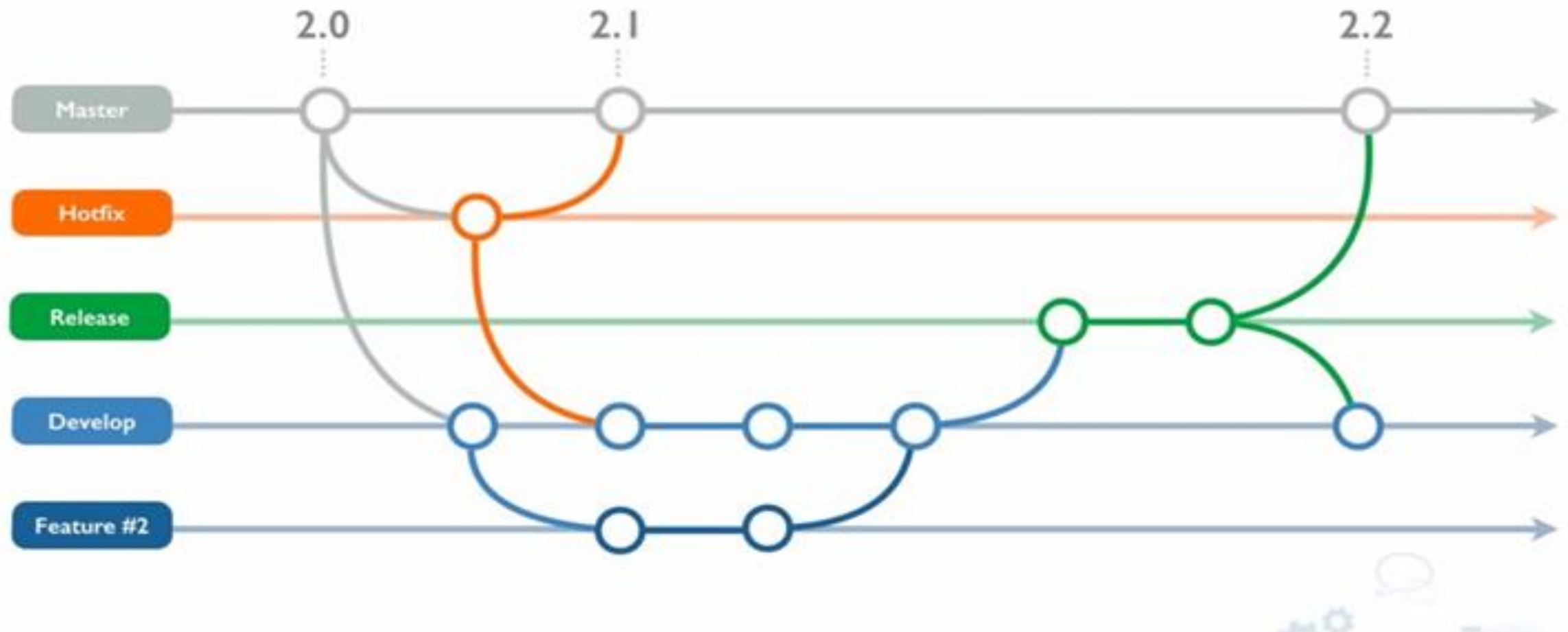
Git Flow



Branchs de apoio

- *Feature Branches*: São criadas a partir da “*develop*” para desenvolver novos recursos. Depois de concluído, o código nessas ramificações é mesclado para “*develop*” e depois excluído.
- *Release Branches*: São criadas a partir da “*develop*” quando uma nova versão deve ser preparada. Os testes e correções finais acontecem aqui antes de mesclá-los nas ramificações “*master**” e “*develop*”.
- *Hotfix Branches*: São criadas a partir da “*master*” apenas para corrigir rapidamente incidentes críticos detectados no ambiente de produção. Estes são então mesclados nas ramificações “*master*” e “*develop*”
- * o termo mais usado agora é *main*

Git Flow



GitHub Flow



- É um fluxo de trabalho Git simples e leve projetado para entrega contínua e colaboração rápida.
- É ideal para iniciantes e equipes que desejam lançar atualizações com frequência.
- Ele incentiva atualizações pequenas e rápidas e fácil colaboração, tornando-o ideal para projetos com lançamentos rápidos.

GitHub Flow



Passos:

1. **Main Branch:** Há apenas uma ramificação principal (geralmente chamada de “*main*”) que contém código pronto para produção.
2. **Criar uma Branch:** Os desenvolvedores criam uma nova ramificação de “*main*” para cada recurso (*feature*) ou correção em que trabalham.
3. **Trabalho direto na Branch Feature:** As alterações são feitas e confirmadas na ramificação criada.

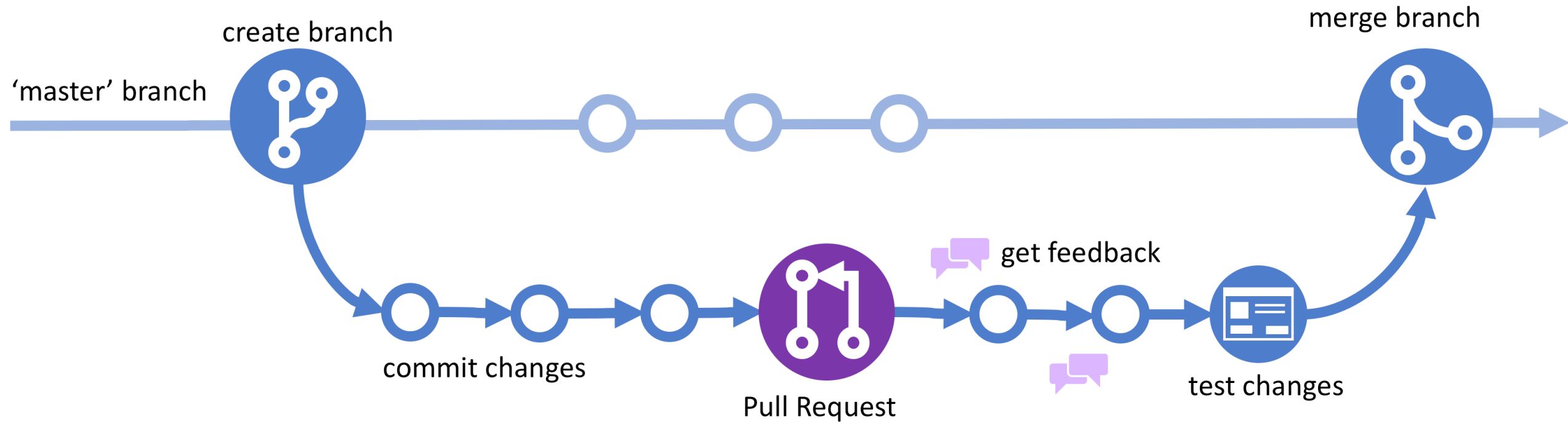
GitHub Flow



Passos:

4. **Cria um *Pull Request*:** Quando o trabalho estiver pronto, os desenvolvedores criarão uma solicitação de *pull* para mesclar suas alterações de volta em "*main*".
5. **Revisão do Código:** Os membros da equipe revisam o *Pull Request*, testam e sugerem alterações, se necessário.
6. **Merge to Main:** Depois de obter a aprovação do *Pull Request*, a *Branch* é mesclada com a "*main*" e pode ser implantado.

GitHub Flow



GitLab Flow



- Combina a simplicidade do GitHub *Flow* com ferramentas adicionais para gerenciar diferentes ambientes (como Desenvolvimento, Preparo e Produção).
- Concentra-se em alinhar o processo de desenvolvimento com o processo de implantação.
- É ideal para equipes que precisam de fluxos de trabalho estruturados e implantações automatizadas em vários ambientes.

GitLab Flow



Pontos Chave:

- *Main Branch*: A branch “main” ou “master” contém o código pronto para produção.
- *Feature Branches*: Os desenvolvedores criam ramificações de *features* para trabalhar em novos recursos ou correções. Depois de concluídos, eles são mesclados de volta ao “branch main”.

GitLab Flow



Pontos Chave

- **Environment Branches:** O GitLab *Flow* oferece suporte a vários ambientes (como "Desenvolvimento", "Preparação", "Pré-produção") nos quais o código é implantado e testado antes de ser liberado para produção.
- **Merge Requests:** As alterações de código são revisadas por meio de solicitações de mesclagem ou solicitações de *pull* (*pull requests*).

GitLab Flow

